

64 通道 64GSa/s TIADC 校准技术

从 Verilog ASIC 到 SoC C 语言实现

GLC Knowledge Base

TIADC 校准项目组

May 18, 2026

目录

- 1 背景与问题
- 2 三步校准算法
- 3 性能评估
- 4 从 ASIC 到 SoC
- 5 计算复杂度与实时性
- 6 推荐架构与 C 实现
- 7 方案对比
- 8 总结

为什么需要 TIADC?

单 ADC 的瓶颈

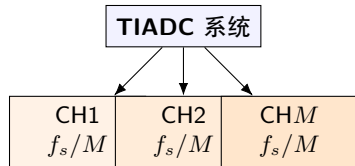
- 工艺限制: CMOS 晶体管特征频率有限
- 功耗墙: 高速 ADC 功耗随采样率超线性增长
- 精度-速度矛盾: 高位宽与高速度难以兼得

TIADC 的解决思路

M 个低速率子 ADC 并行工作:

每个以 f_s/M 采样, 相位依次错开 $1/f_s$

⇒ 等效采样率 f_s , 为单通道的 M 倍



× M 采样率提升
本报告: 64 通道, 64 GSa/s
 $T_s \approx 15.6$ ps

失配问题——理想与现实的差距

核心矛盾

理论上 M 个完全一致的子 ADC 能完美并行。

但工艺偏差 (**Process Variation**) 导致各通道不可避免的失配。

不校准则 **TIADC** 动态性能不如单个子 **ADC**!

失调失配 (Offset)

$$y_k = x_k + O_k$$

加法性误差

与输入频率无关

增益失配 (Gain)

$$y_k = (1 + \Delta G_k)x_k$$

乘法性误差

与信号幅度成正比

时钟偏斜 (Timing Skew)

$$\epsilon \propto f_{in} \cdot \Delta t$$

正比于输入频率

高频时影响最大!

三种失配均在 f_s/M 整数倍处引入杂散 $\rightarrow$ 严重劣化 SFDR / SNR

三种失配对比总结

特性	Offset	Gain	Timing Skew
物理根源	偏置电压不匹配	参考电压/器件不匹配	时钟分配不匹配
数学形式	加法 $+O_k$	乘法 $\times G_k$	时间位移 Δt_k
频谱位置	f_s/M 整数倍	f_s/M 整数倍	f_s/M 整数倍
频率关系	无关	$\propto$ 信号幅度	$\propto$ 信号频率
校准难度	★★	★★	★★★
校准顺序	第 1 步	第 2 步	第 3 步

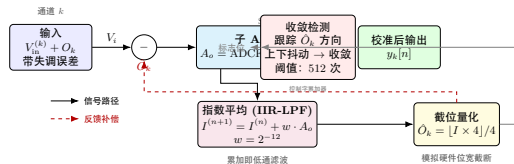
第一步：Offset 校准——指数平均法

核心公式（一阶 IIR 低通）

$$\bar{x}(n) = \bar{x}(n-1) + \alpha[x(n) - \bar{x}(n-1)]$$

- 每次采样都更新，输出连续
- 硬件：加法器 + 移位寄存器，无需乘法器
- $\alpha = 2^{-10} \sim 2^{-21}$ （8 档自动切换）

TIADC 失调校准 (Offset Calibration)



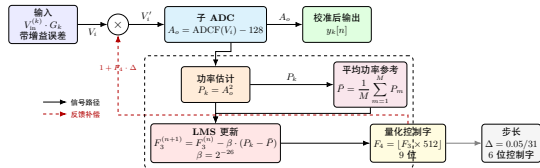
第二步：Gain 校准——LMS 功率均衡

LMS 迭代公式

$$g'_i[n+1] = g'_i[n] + \alpha(T_h - y_i^2[n])$$

- 目标：消除通道间相对增益误差
- 比较各通道功率与参考功率
- 参考值可选：通道 1 功率 / 所有通道均值
- $\alpha = 2^{-32} \sim 2^{-39}$ （极精细步长）

TIADC 增益校准 (Gain Calibration)



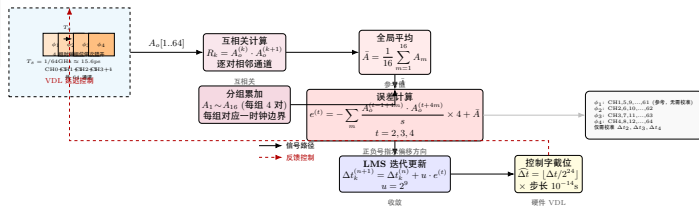
第三步: Timing Skew 校准——自相关法

自相关差值

$$D_{\Delta t} = R_x(T_{ck} - \Delta T) - R_x(T_{ck} + \Delta T)$$

- 利用相邻通道采样的互相关
- $D_{\Delta t}$ 符号 $\rightarrow$ 偏移方向
- $D_{\Delta t}$ 大小 $\rightarrow$ 偏移程度
- 64 通道 $\rightarrow$ 4 组时钟, 校准 3 个参数

TIADC 时钟偏斜校准 (Timing Skew Calibration)



综合校准流水线 & 校准效果

不可调换的顺序

- ① **Offset** — 消除直流偏置
- ② **Gain** — 均衡通道功率
- ③ **Timing** — 对齐采样时刻

all_cali.m 实现三重嵌套

指标	校准前	校准后	改善
SNR	~25 dB	~48 dB	+23 dB
SFDR	~35 dBc	~60 dBc	+25 dB
ENOB	~4 bits	~7.5 bits	+3.5 bits

FFT 分析流程

去直流 → 加窗 → FFT → 搜索基频
→ 搜索谐波 (2~9 次) → 计算指标

- **SNR**: $10 \log(P_{sig}/P_{noise})$
- **SINAD**: $10 \log(P_{sig}/(P_{noise} + P_{dist}))$
- **SFDR**: 信号与最大杂散差距——最直观
- **ENOB**: $(SINAD - 1.76)/6.02$

关键函数

`ADC_dynamic_with_figure.m`

- 输入: ADC 输出数据
- 输出: SNR, SINAD, SFDR, ENOB, THD
- 自动绘制标注频谱图
- 校准前/后对比验证

范式转换: ASIC $\rightarrow$ SoC

原始 ASIC 方案 (Verilog)

- 全硬件并行: 每通道独立运算单元
- 定点数: 位宽严格 (29 位 / 9 位)
- 硬件状态机: α 档位切换
- 门级溢出保护
- 流片后不可改

目标 SoC 方案 (PL + PS)

- **PL (FPGA):** 实时数据通路
 - 复用 Verilog / 新写 HLS C
 - 全并行定点运算
- **PS (ARM):** 控制与监测
 - C 语言灵活策略
 - 在线 FFT / 日志 / OTA

核心问题: C 语言能否替代 Verilog 实现实时校准?

关键洞察：参数估计 $\neq$ 参数应用

校准参数应用（必须高速）

- 每个采样点减去 offset
- 乘以 gain 补偿系数
- 调整时钟相位
- 必须 @ 64 GHz
- 运算极轻：1 次加减 +1 次移位

⇒ **FPGA 硬连线**

校准参数估计（可降采样）

- 从数据估算 offset/gain/skew
- 失配源：温度漂移 + 老化
- 时间常数：秒 ~ 分钟
- 降采样 1024:1 完全可行
- 更新率：kHz~MHz 级

⇒ **C 代码在 ARM 上**

核心结论

失配参数变化是秒到分钟尺度的，完全不需要每 15.6 ps 重新估计！
降采样 1024 倍后，C 代码的实时性约束从 15.6 ps $\rightarrow$ 16 μ s，完全可行。

降采样后 C 代码 CPU 负载

任务	有效输入率	运算量/次	CPU 负载 @1.2GHz
Offset IIR (64 通道)	62.5 kHz	~5 指令/通道	< 0.02%
Gain LMS (64 通道)	62.5 kHz	~10 指令/通道	< 0.04%
Timing 自相关	~1 MHz	~200 指令	< 0.02%
FFT 监测 (16K 点)	10 Hz	~500K 指令	< 0.5%
总计			< 1% CPU

不同场景对高采样率估计的需求

场景	需要 64GHz 估计?	原因
芯片出厂校准	×	一次性, 可用外部仪器
上电启动校准	×	毫秒级收敛即可
温度漂移跟踪	×	温度变化是秒级
实时数据补偿	✓	但这是「应用」不是「估计」

计算复杂度分析：每采样运算量

校准	乘法/通道	加法/通道	×64 通道	频率
Offset	2	2	128 mul + 128 add	每采样
Gain	4	3	256 mul + 192 add	每采样
Timing	~0.017	~0.034	~1.1+2.2	每 64 采样
总计	~385 mul + ~322 add			

时间预算

64 GSa/s: $T_s = 1/(64 \times 10^9) \approx 15.6 \text{ ps}$ ——极为苛刻！

实时性：纯 C 语言可行吗？

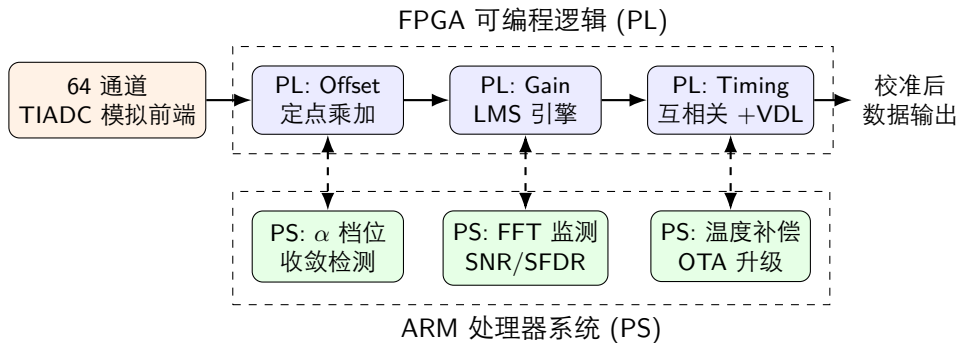
方案	每采样 CPU 周期	结论
64 GSa/s + A53 @1.2GHz	15.6ps/0.8ns $\approx$ 0.02	完全不可行
16 GSa/s + A53 @1.2GHz	62.5ps/0.8ns $\approx$ 0.08	不可行
1 GSa/s + A53 @1.2GHz	1ns/0.8ns $\approx$ 1.3	勉强，需 SIMD
500 MSa/s + Cortex-M7	$\sim$ 30 周期/采样	可行 $\checkmark$
推荐：PL + PS 混合	PL 并行 $\gg$ CPU 串行	唯一可行方案

结论

纯 C 语言仅对 ≤ 500 MSa/s 系统可行。

64 GSa/s 必须用 **FPGA (PL)** 硬件加速 + **ARM (PS)** C 语言控制。

推荐 SoC 架构



分工原则

PL: 确定性实时运算（纳秒级）

PS: 策略性灵活控制（毫秒级）

C 语言定点实现：Offset 校准

```
#define M_CHANNELS    64
#define ALPHA_SHIFT   12           //  $\alpha = 2^{-12}$ 

typedef struct {
    int32_t accum;                // 29-bit accumulator (Q4.25)
    int8_t  offset_est;          // 8-bit offset estimate
} offset_calib_t;

static inline int16_t offset_calibrate(uint8_t ch, int16_t adc_raw) {
    offset_calib_t *ctx = &offset_ctx[ch];
    // Exponential averaging:  $\text{accum} += \alpha * \text{adc\_raw}$ 
    ctx->accum += (int32_t)adc_raw >> ALPHA_SHIFT;
    // Truncation: extract bits [28:21]
    ctx->offset_est = (int8_t)((ctx->accum >> 21) & 0xFF);
    return adc_raw - ctx->offset_est;
}
```

仅需 1 次移位 + 1 次加法 + 1 次减法——可用移位替代乘法

C 语言实现：Timing Skew 校准

```
#define N_GROUPS 4                // 4 clock groups, group 1 = reference
timing_calib_t timing_ctx[N_GROUPS];
static int16_t sample_buf[64];

void timing_skew_calibrate(void) {    // Called every 64 samples
    int32_t A[16] = {0};
    for (int m = 0; m < 16; m++) {    // 16 cross-correlation groups
        int base = m * 4;
        A[m] = (int32_t)sample_buf[base+0]*sample_buf[base+1]
            + (int32_t)sample_buf[base+1]*sample_buf[base+2]
            + (int32_t)sample_buf[base+2]*sample_buf[base+3]
            + (int32_t)sample_buf[base+3]*sample_buf[(base+4)%64];
    }
    int32_t avg = 0;
    for (int m = 0; m < 16; m++) avg += A[m];
    avg >>= 4;                        // Global average

    for (int g = 1; g < 4; g++) {    // Calibrate groups 2,3,4
        int32_t corr_sum = 0;
        for (int m = 0; m < 16; m++)
            corr_sum += (int32_t)sample_buf[g-1+m*4] * sample_buf[(g+m*4)%64];
        timing_ctx[g].skew_accum += (int64_t)(avg - corr_sum) << 9; // LMS
        timing_ctx[g].skew_ctrl = (int32_t)(timing_ctx[g].skew_accum >> 24);
    }
}
```

C 语言实现: α 档位切换与收敛检测

```
alpha_level_t check_convergence(converge_detector_t *det, int32_t est) {
    if (est > det->last_est) det->up_count++;
    else if (est < det->last_est) det->down_count++;
    det->last_est = est;

    if (++det->sample_cnt >= 512) {
        int32_t diff = abs(det->up_count - det->down_count);
        if (diff < 16) {           // < 3% deviation => converged
            switch (det->level) {
                case ALPHA_COARSE:  det->level = ALPHA_MEDIUM; break;
                case ALPHA_MEDIUM:  det->level = ALPHA_FINE;   break;
                case ALPHA_FINE:    break; // Finest level reached
            }
        }
        det->up_count = det->down_count = det->sample_cnt = 0;
    }
    return det->level;
}
```

档位设计

Offset: $2^{-10} \rightarrow 2^{-12} \rightarrow 2^{-14} \rightarrow \dots \rightarrow 2^{-21}$ (8 档)

Gain: $2^{-32} \rightarrow 2^{-33} \rightarrow \dots \rightarrow 2^{-39}$ (8 档)

Timing: $2^{-8} \rightarrow 2^{-9} \rightarrow \dots \rightarrow 2^{-15}$ (8 档)

ASIC vs SoC 全面对比

维度	ASIC (Verilog)	SoC (PL + C/PS)
实现方式	Verilog 硬件描述	PL Verilog/HLS + PS C
并行度	全并行	PL 全并行, PS 串行
α 档位切换	硬件状态机	C 灵活控制
收敛检测	逻辑门比较	C 灵活统计
FFT 监测	不支持	在线软件 FFT
温度补偿	需额外硬件	软件查表
开发周期	长 (流片 ~ 月)	短 (FPGA 快迭代)
修改灵活性	低 (重新流片)	高 (软件更新)
量产成本	低	较高
功耗	低	较高

C 语言的独特价值

在线分析与自适应

- 周期性 FFT 分析校准后数据
- 实时追踪 SNR / SFDR / ENOB
- 卡尔曼滤波预测参数漂移
- 统计方法优化收敛策略

运维与迭代优势

- OTA 固件升级校准算法
- 远程调试与参数调优
- 温度-参数 LUT 维护
- 日志记录与故障回溯

开发流程建议

MATLAB 仿真 → HLS C 综合 → Zynq/Kria 验证 → PS C 控制 → 迭代优化

总结与展望

三步校准——理论到实践

Offset (指数平均) → Gain (LMS 功率均衡) → Timing (自相关 + LMS)

SoC 方案核心结论

- 纯 C 不可行 (≥ 1 GSa/s)
- PL + PS 混合是唯一可行方案
- PL 处理实时数据通路
- PS C 语言负责控制策略

未来改进方向

- 多档位 α 自适应
- 后台校准 (不中断数据流)
- 温度漂移动态补偿
- 盲校准技术
- 轻量 ML 预测校准

PL 做确定性实时运算 + PS C 做策略性灵活控制

最大限度发挥两种计算范式的优势

谢谢!

64 通道 64GSa/s TIADC 校准系统

从 Verilog ASIC 到 SoC C 语言实现

完整报告见 `tiadc_cali_soc.pdf`